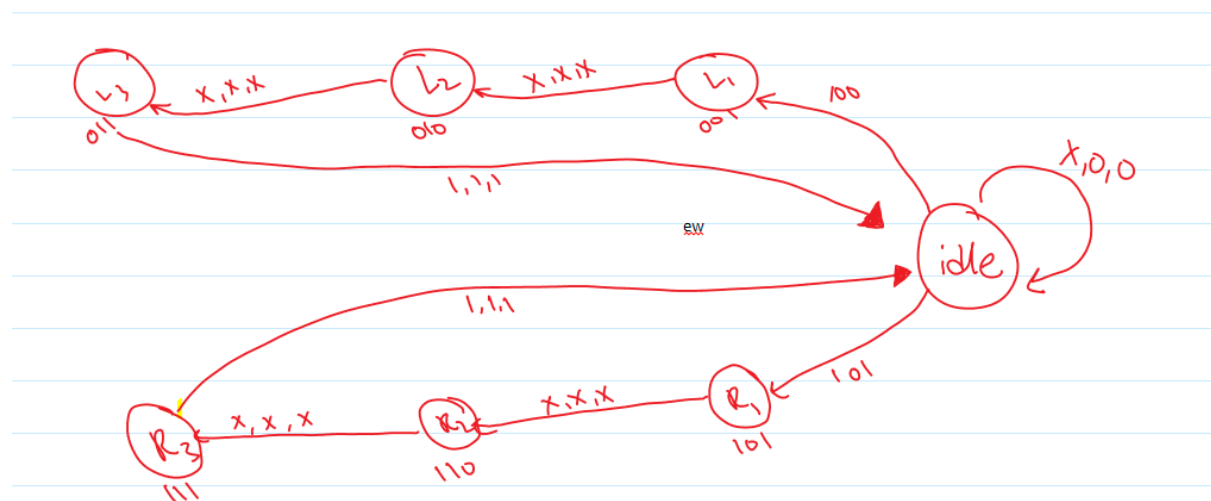


Lab 12 – Worksheet

Name: Sarosh mehdi, Sualeh abbas, Hamza hasan, Ghazi Hussain	ID: 10314, 10303, 10346, 10345	Section: T2
--	--------------------------------	-------------

i. State Transition Diagram

Sketch the state transition diagram. Indicate the input conditions that cause transitions among states. Use the signals 'left' and 'right' for the direction indicator.



ii. State Assignment

Along with your state transition diagram, pick any state assignment method to map symbolic states to binary values.

IDLE → 000

L1 → 001

L2 → 010

L3 → 011

R1 → 101

R2 → 110

R3 → 111

Construct state transition table based on state diagram and chosen state assignment. Add columns for mapping the states to output.

[illegible]

iv. Circuit Diagram of FSM

Obtain input and output equations and sketch circuit diagram to implement the FSM (a sample of Mealy FSM is assuming the D-flip flop as memory element is given in Figure 12.1 as a reference)

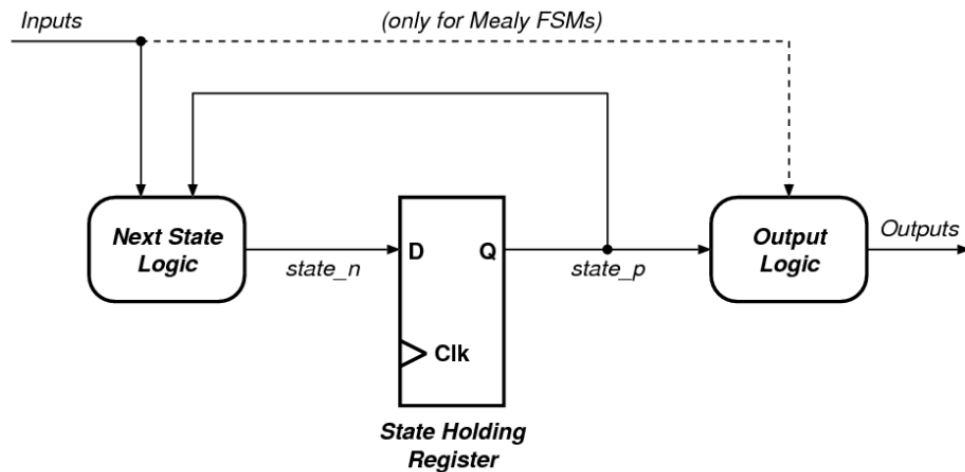
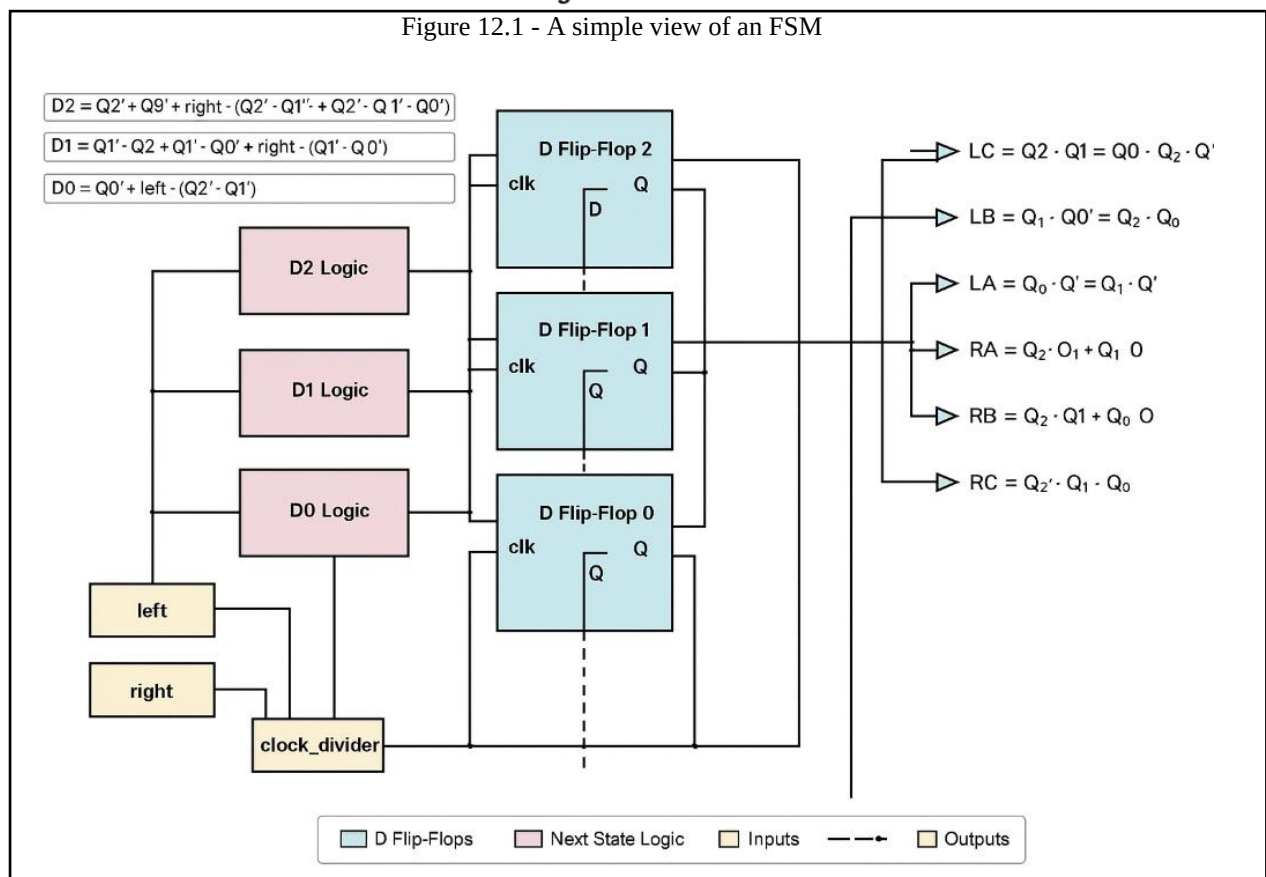


Figure 12.1 - A simple view of an FSM



Output Equations:

LC=A'.B.C
 LB=A'.B
 LA=A'.C
 RA=A
 RB=A.C
 RC=A.B

v. *Verilog Implementation*

On Eda Playground or Vivado create a new project Add a new Verilog source and implement the finite state machine using what you have learned in the lecture about Verilog. Be sure to include a clock input (clk) and a reset (rst) in your circuit.

*Provide appropriately **commented** code here*

```
`timescale 1ns / 1ps
////////////////////////////////////////////////////////////////
// Module Name: tail_light_fsm
// Description: Moore FSM to control 1965 Ford Thunderbird tail lights
////////////////////////////////////////////////////////////////

module tail_light_fsm (
    input clk,
    input reset,
    input left,
    input right,
    input enable,          // Enable signal from clock divider
    output reg LC, LB, LA,
    output reg RA, RB, RC
);

    // State encoding
    parameter IDLE = 3'b000,
               L1  = 3'b001,
               L2  = 3'b010,
               L3  = 3'b011,
               R1  = 3'b100,
               R2  = 3'b101,
               R3  = 3'b110;

    reg [2:0] state, next_state;

    // State register with synchronous reset
    always @(posedge clk or posedge reset) begin
```

```
    if (reset)
        state <= IDLE;
    else if (enable)
        state <= next_state;
    end

// Next-state logic
always @(*) begin
    case (state)
        IDLE: begin
            if (left)
                next_state = L1;
            else if (right)
                next_state = R1;
            else
                next_state = IDLE;
        end
        L1: next_state = L2;
        L2: next_state = L3;
        L3: next_state = IDLE;
        R1: next_state = R2;
        R2: next_state = R3;
        R3: next_state = IDLE;
        default: next_state = IDLE;
    endcase
end

// Output logic (Moore: depends only on state)
always @(*) begin
    // Default off
    LA = 0; LB = 0; LC = 0;
    RA = 0; RB = 0; RC = 0;

    case (state)
        L1: LA = 1;
        L2: begin LA = 1; LB = 1; end
        L3: begin LA = 1; LB = 1; LC = 1; end
        R1: RA = 1;
        R2: begin RA = 1; RB = 1; end
        R3: begin RA = 1; RB = 1; RC = 1; end
        default: ; // IDLE or invalid state
    endcase
end

endmodule
```

vi. *Implementing the Clock*

The clock available on Basys-3 board is 100 MHz, means that every clock period is 10 nanoseconds long. The entire blinking sequence would be then 40 ns. This is a very short time. light travels less than 250 meters during that time. If we want to see our sequence, we need to find a way to slow down the circuit. This can be achieved in two ways. We can either implement a clock divider circuit that divides the clock by a few million times, or we can generate an enable signal every few million cycles, and then use this enable signal to control our next state transition. Refer to Lab 10 to design a clock divider of your desired value

Provide appropriately **commented** code of clock div

```
`timescale 1ns / 1ps
/////////////////////////////////////////////////////////////////
// Module Name: clock_divider
// Description: Divides 100MHz Basys-3 clock to a slower pulse for FSM transitions
/////////////////////////////////////////////////////////////////

module clock_divider #(parameter COUNT_MAX = 100_000_000)(
    input clk,
    input reset,
    output reg enable
);

    reg [26:0] counter = 0;

    always @(posedge clk or posedge reset) begin
        if (reset) begin
            counter <= 0;
            enable <= 0;
        end else if (counter == COUNT_MAX - 1) begin
            counter <= 0;
            enable <= 1; // Send a single-cycle enable pulse
        end else begin
            counter <= counter + 1;
            enable <= 0;
        end
    end
end

endmodule
```

vii. Top Level Module

Write top module to implement FSM along with clock divider

Provide appropriately **commented** code for designed module

```
// Top-level module for controlling tail lights on a Basys-3 board
module top_module (
    input clk,          // 100MHz clock input from the Basys-3 board
    input reset,        // Active-high reset signal
    input left,         // Signal to activate left turn lights
    input right,        // Signal to activate right turn lights
    output LC, LB, LA, // Outputs for left turn lights (Center, B, A)
    output RA, RB, RC // Outputs for right turn lights (A, B, Center)
);

    wire enable;        // Enable signal generated by the clock divider to control the FSM speed

    // Instantiate clock divider
    // This module divides the 100MHz clock to a lower frequency to slow down the FSM
    // transitions,
    // making the tail light effect visible to the human eye
    clock_divider #(100_000_000) clkdiv (
        .clk(clk),      // Input clock (100 MHz)
        .reset(reset),  // Reset signal
        .enable(enable) // Output enable signal (slower clock pulse)
    );

    // Instantiate the finite state machine (FSM) for tail lights
    // This FSM handles the light sequencing based on the left/right input
    tail_light_fsm fsm (
        .clk(clk),      // Input clock
        .reset(reset),  // Reset signal
        .left(left),    // Input to activate left light sequence
        .right(right),  // Input to activate right light sequence
        .enable(enable), // Enables FSM state transition when high

        // Outputs to control each individual tail light
        .LC(LC), .LB(LB), .LA(LA), // Left side tail lights
        .RA(RA), .RB(RB), .RC(RC)  // Right side tail lights
    );

endmodule
```

viii. Define the constraints (to create .xdc file)

Define constraints for your module to implement on Basys-3 Board

Add Constraint file here

```
set_property PACKAGE_PIN W5 [get_ports clk]
set_property PACKAGE_PIN L1 [get_ports LA]
set_property PACKAGE_PIN P1 [get_ports LB]
set_property PACKAGE_PIN N3 [get_ports LC]
set_property PACKAGE_PIN U19 [get_ports RA]
set_property PACKAGE_PIN E19 [get_ports RB]
set_property PACKAGE_PIN U16 [get_ports RC]
set_property PACKAGE_PIN U18 [get_ports reset]
set_property PACKAGE_PIN W19 [get_ports left]
set_property PACKAGE_PIN T17 [get_ports right]
set_property IOSTANDARD LVCMOS33 [get_ports clk]
set_property IOSTANDARD LVCMOS33 [get_ports LA]
set_property IOSTANDARD LVCMOS33 [get_ports LB]
set_property IOSTANDARD LVCMOS33 [get_ports LC]
set_property IOSTANDARD LVCMOS33 [get_ports left]
set_property IOSTANDARD LVCMOS33 [get_ports RA]
set_property IOSTANDARD LVCMOS33 [get_ports RB]
set_property IOSTANDARD LVCMOS33 [get_ports RC]
set_property IOSTANDARD LVCMOS33 [get_ports reset]
set_property IOSTANDARD LVCMOS33 [get_ports right]
```

ix. Test on BASYS 03 Board

Program Basys-3 board to test and validate your designed modules

RA Signature

b. Assessment Rubric

Marks Distribution:

		LR2 Code/Model	LR5 Results	LR11 Design	LR7 Viva	LR9 Report
In-Lab	Task 1	10			10	10
	Task 2	10				
	Task 3	10				
	Task 4			10		
	Task 5	10				
	Task 6	10				
	Task 7	5				
	Task 8	5				
	Task 9		10			
Total						100 Points

Marks Obtained:

		LR2 Code/Model	LR5 Results	LR11 Design	LR7 Viva	LR9 Report
In-Lab	Task 1					
	Task 2					
	Task 3					
	Task 4					
	Task 5					
	Task 6					
	Task 7					
	Task 8					
	Task 9					
Total						